DuocUC | ONLINE



DESARROLLO BACKEND III (PBY2203)

Informe tecnico

Modernizacion del core bancario del Banco XYZ

Migracion batch, patron Backend for Frontend y microservicios resilientes

Agustin Andrews V.

Desarrollo Backend III (PBY2203)

Evaluacion Final Transversal - Semana 9

Indice

1. Resumen ejecutivo

2. Los cinco procesos clave de la migración

- 2.1 Migración de los procesos batch a Spring Batch
- 2.2 División del sistema monolítico en microservicios
- 2.3 Implementación del patrón Backend for Frontend
- 2.4 Seguridad distribuida con Spring Cloud Security
- 2.5 Integración de mensajería asíncrona con Apache Kafka

3. Requerimientos del negocio y propuesta de arquitectura

- 3.1 Requerimiento 1: continuidad operativa ante fallos parciales
- 3.2 Requerimiento 2: eficiencia por canal sin multiplicar el nucleo
- 3.3 Requerimiento 3: escalar lo que lo necesita, cuando lo necesita

4. Arquitectura de la solución

- 4.1 Componentes y responsabilidades
- 4.2 Cómo se integran entre sí

5. Parte 1: migración de los procesos batch

- 5.1 Calidad de los datos de entrada
- 5.2 Resultado de la ejecución
- 5.3 Manejo de errores, reintentos y paralelismo
- 5.4 Equivalencia con el sistema heredado

6. Parte 2: el patrón Backend for Frontend

- 6.1 Las tres formas de la misma información
- 6.2 Autenticación y autorización especificas por canal
- 6.3 Reglas propias del canal

7. Parte 3: microservicios resilientes

- 7.1 La saga coreografiada
- 7.2 Garantías sobre los eventos
- 7.3 Tres consumidores del mismo evento
- 7.4 Los cinco patrones de Resilience4j

8. Casos de uso

9. Despliegue y escalabilidad

9.1 Contenedores

9.2 Orquestación

9.3 Escalabilidad horizontal

10. Resultados y comparación con el sistema heredado

10.1 Verificación

11. Desafios enfrentados y soluciones

11.1 El cortocircuito que no se abria

11.2 Los esquemas que borraban los datos migrados

11.3 El balanceo que existía pero no se veia

12. Propuestas de mejora y próximos pasos

12.1 Camino de despliegue en la nube

13. Conclusión

1. Resumen ejecutivo

El Banco XYZ es una institución financiera con más de treinta años en el mercado cuyo sistema de gestión opera sobre una plataforma heredada escrita en COBOL y guiones de shell, ejecutandose sobre servidores mainframe. Ese sistema sostiene operaciones críticas: el procesamiento de transacciones, la generación de estados de cuenta, el cálculo de intereses y la gestión de clientes. Las limitaciones de escalabilidad, el costo de mantenimiento y la dificultad para integrar tecnologias nuevas motivaron la decisión de modernizar la infraestructura.

Este informe documenta la solución construida: una plataforma de quince contenedores que reemplaza al sistema heredado con una arquitectura de microservicios desplegable en la nube. La solución cubre las tres partes que plantea el caso, y lo hace como un sistema único y no como tres trabajos independientes: el proceso batch migra los datos del legado y los deja saneados en las bases de los microservicios, los microservicios los sirven y tres Backend for Frontend los adaptan a cada canal.

Cifras de la solución entregada

Dimensión	**Valor**
Módulos Maven	14, en un único reactor
Contenedores orquestados	15 (12 imagenes propias más PostgreSQL, Kafka y su panel)
Procesos batch migrados	3, con particionado en 8 hilos
Microservicios de negocio	3, más un consumidor de eventos
Canales con backend propio	3 (web, móvil y cajeros automáticos)
Tópicos de Kafka	4
Pruebas automatizadas	73, todas en verde
Registros del legado procesados	3.000 filas de tres archivos CSV

La totalidad de lo que se afirma en este informe está respaldada por evidencia reproducible. Las cinco salidas de consola del directorio evidencia/consola se regeneran ejecutando los guiones correspondientes sobre el sistema en marcha, y las cifras que aparecen aquí provienen de esas ejecuciones y no de estimaciones.

2. Los cinco procesos clave de la migración

Antes de diseñar nada fue necesario identificar qué procesos críticos debía cubrir la migración. Son cinco, y cada uno responde a una limitación concreta del sistema heredado. El orden no es arbitrario: cada proceso depende de que los anteriores estén resueltos.

2.1 Migración de los procesos batch a Spring Batch

El sistema heredado genera tres informes mediante procesos batch nocturnos: el reporte de transacciones diarias, el cálculo de intereses y la generación de estados de cuenta anuales. Están escritos como guiones de shell que procesan archivos planos de forma secuencial y que se detienen ante el primer registro defectuoso, obligando a intervención manual.

Es el primer proceso porque sin el no hay datos. Los microservicios que se construyan después no tienen nada que servir hasta que la información del legado esté migrada y saneada.

2.2 División del sistema monolítico en microservicios

El backend actual es un único proceso que concentra cuentas, pagos y clientes. Un cambio en cualquiera de esos dominios obliga a desplegar todo, y un fallo en uno afecta a los tres. La división se hizo por propiedad de los datos: cada microservicio es dueño exclusivo de su porción del dominio y de su base de datos.

2.3 Implementación del patrón Backend for Frontend

Los tres frontales -navegador, aplicación móvil y cajeros automáticos- se conectan hoy al mismo backend y reciben exactamente la misma respuesta. El teléfono recibe datos que no muestra y el navegador debe hacer varias llamadas para armar una pantalla. Un único backend no puede optimizar para los dos a la vez.

2.4 Seguridad distribuida con Spring Cloud Security

En el monolito la autenticación es un módulo interno. Al repartir el sistema en doce procesos, cada uno necesita saber si quien lo llama está autorizado, sin que cada uno gestione contrasenas. La solución es un único servidor de autorización OAuth 2.0 que emite tokens firmados, y servicios que se limitan a verificar la firma localmente.

2.5 Integración de mensajería asíncrona con Apache Kafka

Una llamada HTTP directa obliga a que el destinatario esté vivo en ese instante y a que quien llama conozca a todos los interesados. Con mensajería, un servicio públicа un hecho y quien quiera reacciona. Es lo que permite agregar un consumidor nuevo sin tocar al productor, y lo que hace que la caida de un servicio no se propague al que lo precede.

3. Requerimientos del negocio y propuesta de arquitectura

La arquitectura no se eligio por preferencia técnica sino a partir de tres requerimientos del negocio. Cada decisión de diseño que sigue está justificada por uno de ellos.

3.1 Requerimiento 1: continuidad operativa ante fallos parciales

Que pide el negocio

Un banco no puede permitirse que un problema en el módulo de clientes impida consultar un saldo. En el monolito actual, un fallo en cualquier módulo detiene el sistema completo.

Decisión de arquitectura y por que

- Microservicios con base de datos propia, de modo que el fallo quede acotado al dominio afectado.
- Comunicación por eventos siempre que sea posible. La única llamada síncrona entre servicios de negocio es la verificación previa de una cuenta, y está protegida con los cinco patrones de Resilience4j.
- Dos políticas de respaldo opuestas y deliberadas: ms-pagos degrada y acepta la operación cuando ms-cuentas no responde, porque el evento se resolvera después; ms-cuentas rechaza la apertura cuando ms-clientes no responde, porque abrir una cuenta a nombre de alguien sin verificar crea un registro incorrecto que luego hay que deshacer a mano. Distinto riesgo, distinta decisión.

La evidencia del archivo 04-resiliencia.txt muestra el comportamiento medido: con una latencia inducida de cuatro segundos, el límite de tiempo corta a los 3,03 segundos; tras varios fallos el cortocircuito se abre y la siguiente llamada responde en 27 milisegundos sin viajar; y al retirar la latencia el sistema recupera solo, con la transición CLOSED_TO_OPEN, OPEN_TO_HALF_OPEN y HALF_OPEN_TO_CLOSED registrada en la bitácora del actuator.

3.2 Requerimiento 2: eficiencia por canal sin multiplicar el nucleo

Que pide el negocio

Los equipos de frontend deben poder avanzar en paralelo y cada canal debe recibir lo que necesita, ni más ni menos. Hoy el canal móvil consume ancho de banda en datos que no muestra.

Decisión de arquitectura y por que

- Un BFF por canal, independiente y desplegable por separado. Lo que comparten es solo la fontanería HTTP -obtener un token, resolver un nombre por Eureka, deserializar una respuesta-; lo que define al patrón, que es que pide cada canal y qué forma le da, vive entero en cada BFF.
- Un cliente OAuth 2.0 distinto por canal, con credenciales y alcances propios. Comprometer un canal no abre los otros dos.
- Ningún BFF tiene base de datos. Si la tuviera, la personalización por canal se convertiría en tres verdades distintas del mismo saldo.

El resultado medido sobre el mismo titular y el mismo nucleo: el canal web devuelve 5.939 bytes y el móvil 887, es decir un 14,9 por ciento. La diferencia no está en los datos sino en la forma: nombres de campo de una letra, titular abreviado, importes redondeados a entero y tipos codificados con la tabla de equivalencia publicada una sola vez por respuesta.

3.3 Requerimiento 3: escalar lo que lo necesita, cuando lo necesita

Que pide el negocio

El procesamiento de pagos tiene picos que el resto del sistema no tiene. Escalar el monolito completo para absorber un pico de pagos significa pagar por capacidad ociosa en todo lo demas.

Decisión de arquitectura y por que

- Servicios sin estado: ningún servicio guarda sesión, la identidad viaja en el token. Una instancia puede morir y otra tomar su lugar.
- Descubrimiento dinámico con Eureka. El gateway y los BFF resuelven lb://ms-pagos y no una dirección fija, de modo que una réplica nueva se incorpora sola.
- Configuración externalizada en un servidor de configuración. Cambiar el umbral del cortocircuito no exige reconstruir ninguna imagen.

La evidencia del archivo 05-escalado-horizontal.txt muestra tres réplicas de ms-pagos y de ms-clientes atendiendo diez consultas cada una de treinta, sin que hiciera falta modificar código ni configuración.

4. Arquitectura de la solución

El sistema se organiza en cuatro capas, que son también la forma en que el banco razona sobre el: una plataforma que resuelve configuración, descubrimiento, identidad y enrutamiento; los microservicios de negocio; los canales; y la migración batch.

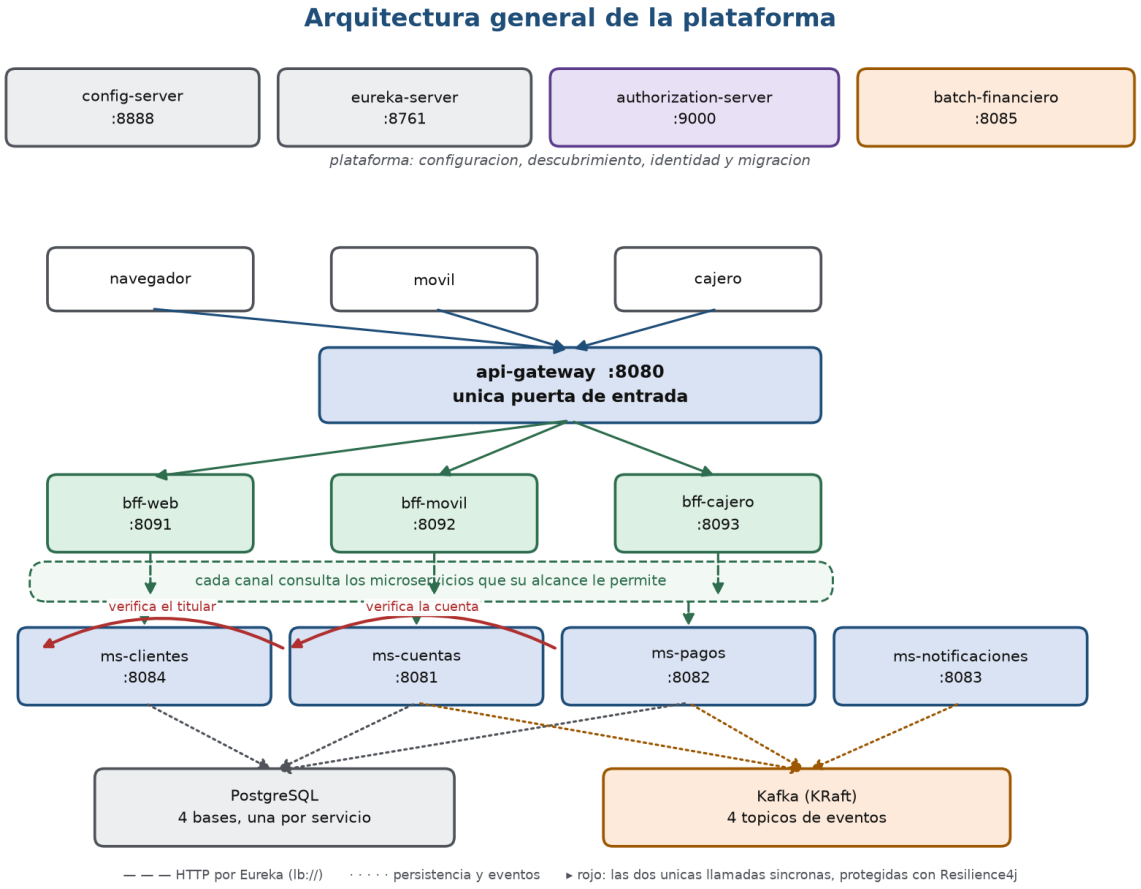


Figura 1. Arquitectura de componentes de la plataforma.

4.1 Componentes y responsabilidades

Componente	Puerto	Responsabilidad
config-server	8888	Entrega la configuración a todos los servicios al arrancar
eureka-server	8761	Registro de instancias vivas para el descubrimiento
authorization-server	9000	Único emisor de tokens OAuth 2.0
api-gateway	8080	Única puerta de entrada; valida el token y enruta
batch-financiero	8085	Migración del sistema heredado; se dispara por REST
ms-clientes	8084	Dueño de la identidad y el perfil del titular

ms-cuentas	8081	Dueño exclusivo del saldo y del ciclo de vida de la cuenta
ms-pagos	8082	Origen de la saga; pagos, transferencias y depósitos
ms-notificaciones	8083	Consumidor puro de eventos; avisos en memoria
bff-web	8091	Canal navegador: vista completa
bff-movil	8092	Canal teléfono: respuesta mínima
bff-cajero	8093	Canal terminal: operaciones críticas, sin datos del titular

4.2 Cómo se integran entre sí

El exterior solo ve el gateway. Toda petición entra por el puerto 8080, donde se valida el token antes de que el tráfico alcance la red interna. El gateway no conoce la dirección de ningún servicio: resuelve nombres lógicos contra Eureka, que es lo que permite que una réplica nueva se incorpore sin configuración.

La validación del token en el gateway no sustituye a la de cada servicio. Cada microservicio vuelve a comprobar el alcance por su cuenta, porque si el gateway fuera el único control bastaría alcanzar un servicio por dentro de la red para saltarselo entero. El gateway responde quién eres; cada servicio responde que puedes hacer aquí.

Entre servicios de negocio la comunicación es por eventos. La excepción son dos verificaciones previas -ms-pagos consultando una cuenta y ms-cuentas consultando un cliente- que son síncronas porque necesitan respuesta inmediata, y que por eso mismo son las únicas protegidas con Resilience4j.

5. Parte 1: migración de los procesos batch

Los tres procesos del sistema heredado se reescribieron con Spring Batch. Cada uno es un job con sus pasos de lectura, procesamiento y escritura, más un paso final que carga el resultado saneado en la base del microservicio que corresponde.

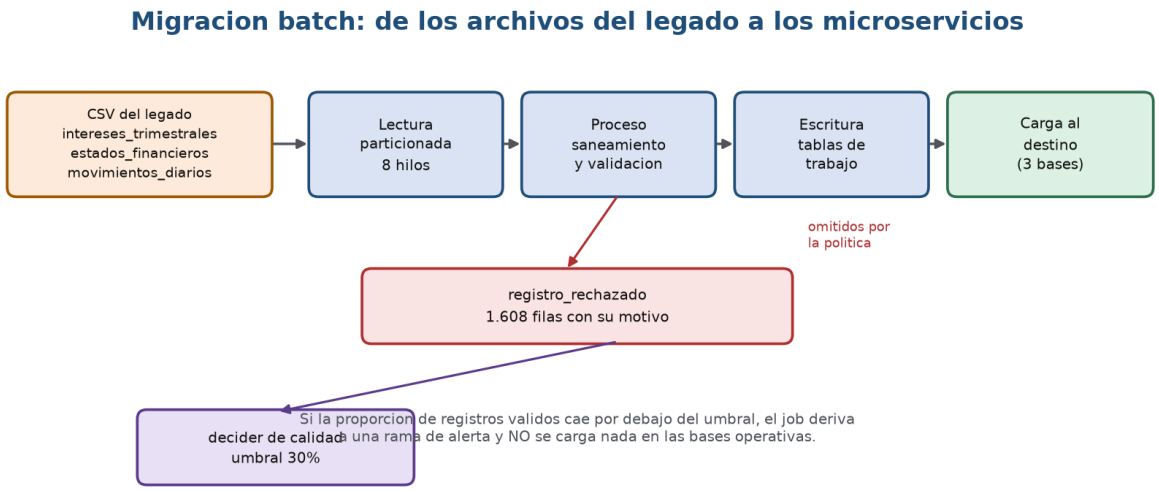


Figura 2. Flujo de los tres procesos batch y su carga en los microservicios.

5.1 Calidad de los datos de entrada

Los archivos del sistema heredado traen defectos intencionales que reproducen los de un legado real. Medidos sobre el conjunto de datos utilizado: cuatro formatos de fecha conviviendo en los tres archivos, 168 montos vacios y 125 negativos en los movimientos diarios, 204 saldos vacios y 171 edades ausentes en el archivo de intereses, 325 tipos de cuenta con marcadores de relleno, y 184 descripciones vacias en los estados anuales.

5.2 Resultado de la ejecución

Proceso	Leidos	Escritos	Omitidos
Calculo de intereses	829	397	603
Estados de cuenta anuales	1.000	604	396
Reporte de transacciones diarias	1.000	392	608

Que se omita más de la mitad de los registros no es un fallo del proceso: es exactamente lo que describe el enunciado cuando señala que los archivos traen errores intencionales. Lo que si

sería un fallo es que el proceso se detuviera ante la primera fila defectuosa, o que las dejara entrar sin avisar. Cada registro omitido queda en una tabla de cuarentena con su motivo: 211 por tipo de transacción fuera de catalogo, 168 por monto ausente, 171 por tipo de cuenta invalido, entre otros.

5.3 Manejo de errores, reintentos y paralelismo

- Política de omisión que clasifica el error en tres familias: defecto del dato, fallo de infraestructura y desconocido. Solo la segunda se reintenta, porque reintentar un dato mal escrito dara siempre el mismo resultado.
- Política de reintento con espera creciente, porque insistir de inmediato contra un recurso saturado solo agrega carga.
- Particionado del paso de carga en ocho hilos, cada uno con su rango del archivo. Medido sobre estos volumenes, fue la configuración optima.
- Un decisor de calidad que deriva el flujo a revisión si la proporción de registros validos cae por debajo del umbral, de modo que un archivo corrupto nunca llega a tocar las bases operativas.

5.4 Equivalencia con el sistema heredado

La migración deja siete clientes, cincuenta cuentas, 604 movimientos historicos y 392 pagos. La identidad de los titulares se deduce agrupando por nombre, que es el único criterio disponible porque el sistema heredado no tiene identificadores de cliente. Es un supuesto, y se documenta como tal: en una migración real este es el punto donde se cruzaría contra el documento de identidad.

El proceso es idempotente. Relanzar la migración completa deja exactamente las mismas cifras, lo que está comprobado en la sección 8 del archivo 01-migración-batch.txt. Eso importa porque un proceso de migración que acumulara sobre lo anterior dejaría cuentas duplicadas al reintentar tras un fallo.

6. Parte 2: el patrón Backend for Frontend

Cada canal tiene un backend propio que consulta el mismo nucleo y entrega una respuesta distinta. La diferencia no está en los datos sino en la decisión de que mostrar y como.

6.1 Las tres formas de la misma información

Canal	Optimiza por	Decisiones de forma
Web	Número de viajes	Agrupa perfil, cuentas, movimientos y pagos en una respuesta; nombres de campo completos; importes con decimales
Móvil	Tamano	Nombres de una letra; titular abreviado; importes a entero; tipos codificados; tres movimientos por cuenta
Cajero	Superficie mínima	Cuatro campos; ningún dato del titular; sin historial; reglas de tope y múltiplo de billete

La comparación medida entre el canal web y el móvil, para el mismo titular, es de 5.939 contra 887 bytes. El canal móvil entrega un 14,9 por ciento de lo que entrega el web.

6.2 Autenticación y autorización especificas por canal

Cada BFF es un cliente OAuth 2.0 distinto, con su propio identificador, su secreto y su conjunto de alcances. No es burocracia: es lo que hace que comprometer un canal no abra los otros dos.

Canal	Alcances concedidos	Lo que deliberadamente NO tiene
canal-web	clientes.leer, cuentas.leer, cuentas.escribir, pagos.leer, pagos.escribir	-
canal-movil	clientes.leer, cuentas.leer, pagos.leer	pagos.escribir: la aplicación muestra saldos; mover dinero pasa por otro canal
canal-cajero	cuentas.leer, pagos.leer, pagos.escribir	clientes.leer: es un equipo en la via pública y no accede al padron de titulares

El token del cajero dura cinco minutos frente a los quince de los otros dos, porque el riesgo de que alguien acceda físicamente al terminal es mayor.

6.3 Reglas propias del canal

El canal de cajeros impone un tope por operación y múltiplos de billete. Estas reglas no viven en el nucleo porque el mismo titular puede retirar más desde una sucursal: son limitaciones del terminal, no del dinero. Se comprueban antes de llamar al nucleo, porque un monto que el cajero no puede entregar físicamente no tiene por que recorrer tres servicios para ser rechazado.

7. Parte 3: microservicios resilientes

7.1 La saga coreografiada

Un pago no se resuelve en una transacción distribuida. ms-pagos registra la operación como pendiente y pública el hecho; ms-cuentas, que es el único dueño del saldo, decide si lo aplica y responde con otro evento; ms-pagos confirma o compensa según lo que llegue.



Figura 3. Saga coreografiada de un pago, con sus dos desenlaces.

No hay orquestador ni bloqueo de dos fases: cada servicio confirma localmente y el acuerdo se alcanza por los eventos. El precio es que existe una ventana en la que el sistema está en un estado intermedio, y por eso el estado PENDIENTE es visible en la API en lugar de esconderse. La API responde 202 y no 201, porque devolver creado cuando el saldo todavía no cambio sería mentirle al canal.

7.2 Garantías sobre los eventos

- Idempotencia en ambos extremos. Kafka entrega al menos una vez, de modo que tras un reequilibrio de particiones el mismo evento puede llegar dos veces. Una tabla de eventos procesados con clave primaria lo impide, y se apoya en la restricción de la base en lugar de una consulta previa porque entre la consulta y la inserción otro consumidor podría colarse.
- Acuse manual al final, ya aplicado el cambio. Con acuse automático, Kafka da el evento por consumido apenas lo entrega y una caida posterior lo perdería.

- La clave del mensaje es el identificador de la operación, lo que fija la partición y garantiza que los eventos de un mismo pago se consuman en orden.
- Confirmación de todas las réplicas en el productor. Es más lento, y es lo correcto cuando el evento representa movimiento de dinero.

7.3 Tres consumidores del mismo evento

El evento de saldo actualizado lo consumen tres servicios con grupos distintos: ms-pagos cierra la saga, ms-clientes recalcula el segmento comercial del titular y ms-notificaciones genera el aviso. Ninguno de los tres fue invocado por nadie, y el productor no sabe que existen. Agregar un cuarto mañana no obliga a tocar nada.

7.4 Los cinco patrones de Resilience4j

Patrón	Modo de fallo que cubre	Configuración
Retry	Fallo pasajero: un paquete perdido, un reinicio de un segundo	3 intentos, espera creciente desde 300 ms
TimeLimiter	Dependencia que acepta la conexión pero no responde	3 segundos
Bulkhead	Que la lentitud del otro agote los hilos propios	10 llamadas concurrentes
RateLimiter	Tumbar al otro servicio cuando vuelve	50 por segundo
CircuitBreaker	Insistir contra algo que está caído	ventana de 10, mínimo 5 llamadas, 50 por ciento de fallos, 10 s abierto

Dos detalles de la configuración cambian el comportamiento de forma no evidente y conviene dejarlos escritos. El primero: el respaldo se declara en la anotación de reintento y no en la del cortocircuito, porque Resilience4j aplica el reintento por fuera, y un respaldo declarado en el cortocircuito convertiría cada fallo en un resultado correcto antes de que el reintento llegara a verlo. El segundo: un recurso inexistente no cuenta como fallo del cortocircuito, porque significa que el otro servicio está sano y respondió lo correcto; si contara, un cliente pidiendo cuentas inexistentes bastaría para dejar sin servicio una dependencia que funciona.

8. Casos de uso

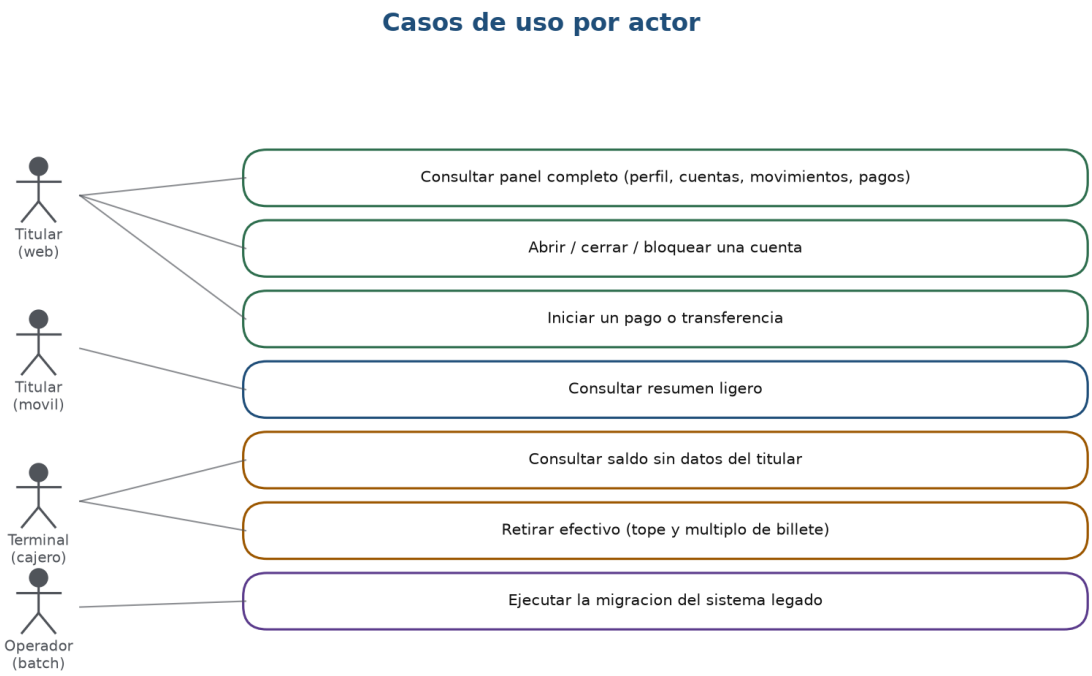


Figura 4. Casos de uso por actor y canal.

Los actores son el titular, que opera desde el navegador o el teléfono; el ejecutivo, que abre y cierra cuentas desde el canal web; el terminal de autoservicio, que se autentica a si mismo y no a una persona; y el operador de la migración, que dispara el proceso batch.

9. Despliegue y escalabilidad

9.1 Contenedores

Un único Dockerfile multietapa produce las doce imagenes, con una etapa final por servicio. La etapa de compilación se construye una sola vez y las doce finales la reutilizan; con doce archivos separados, el reactor Maven se ejecutaría doce veces sobre el mismo código. Cada imagen final copia únicamente su propio jar, de modo que la de un BFF no contiene el código de un microservicio.

Las imagenes de ejecución parten de una JRE y no de un JDK, corren con un usuario sin privilegios y declaran limites de memoria que la máquina virtual respeta. Cada una limita además el número de procesadores que la máquina virtual cree tener, porque doce procesos convencidos cada uno de disponer de la máquina completa dimensionan sus hilos de recolección de basura y de compilación en consecuencia, y el arranque simultaneo satura el equipo.

9.2 Orquestación

El archivo de composición levanta los quince contenedores. Lo más relevante no es la lista de servicios sino el orden: cada uno declara de que depende y espera a que ese otro esté sano, no solo iniciado. La diferencia es práctica, porque un contenedor de PostgreSQL acepta conexiones en el puerto segundos antes de poder atender consultas, y un servicio que arranca en esa ventana muere al intentar crear su esquema.

9.3 Escalabilidad horizontal

Un archivo de superposición permite levantar varias réplicas de los tres microservicios de negocio. Lo único que hace es quitarles el nombre fijo de contenedor y el puerto del anfitrion, que es lo único que impedia tener más de una; el resto del sistema ya estaba preparado.

Servicio	Escala	Motivo
ms-clientes, ms-cuentas, ms-pagos	Sí	Sin estado; el estado está en la base
Los tres BFF	Sí	Sin estado, solo transforman
api-gateway	Si, con un balanceador delante	En la nube lo hace un balanceador gestionado
ms-notificaciones	Con reservas	Guarda avisos en memoria; cada replica tendría los suyos

batch-financiero	No	Es un proceso de un solo uso, no un servicio permanente

La comprobación no es que los contenedores existan, sino que respuestas sucesivas a la misma ruta las conteste una instancia distinta. Con tres réplicas y treinta consultas, el reparto medido fue de diez por instancia.

10. Resultados y comparación con el sistema heredado

Aspecto	Sistema heredado	Solución entregada
Procesamiento batch	Secuencial; se detiene ante el primer error	Particionado en 8 hilos; omite y registra el motivo, continúa
Datos defectuosos	Intervención manual	Cuarentena automatica con el motivo de cada rechazo
Despliegue	Todo o nada	Doce unidades independientes
Fallo de un módulo	Afecta al sistema completo	Acotado; el resto sigue operando con respaldo
Respuesta por canal	La misma para los tres	Adaptada: el móvil recibe el 14,9 por ciento del web
Seguridad	Centralizada en el monolito	OAuth 2.0; alcances por recurso y por canal
Escalado	Vertical, todo el sistema	Horizontal y por servicio
Configuración	En el código	Centralizada; cambia sin reconstruir imagenes
Integración de consumidores	Modificar al productor	Suscribirse a un tópico; el productor no se entera

10.1 Verificación

El proyecto incluye 73 pruebas automatizadas. Las de los microservicios con persistencia corren contra una base real en modo de compatibilidad y utilizan el mismo archivo de esquema que producción, no una copia, porque una copia se desincroniza en silencio y las pruebas acaban validando una estructura que ya no existe. Las de resiliencia levantan un servidor HTTP real al que se le indica como portarse, en lugar de simular las excepciones: un doble de prueba verificaría la configuración sin pasar nunca por la red que esa configuración protege.

11. Desafios enfrentados y soluciones

Tres defectos encontrados durante el desarrollo merecen quedar documentados, porque los tres compartían la misma característica: el sistema parecía funcionar.

11.1 El cortocircuito que no se abria

Síntoma

Con la dependencia caida, el respaldo respondia correctamente y las métricas mostraban cero fallos. El cortocircuito permanecía cerrado indefinidamente.

Causa

La ejecución asíncrona envuelve en una excepción de composición todo lo que lanza su proveedor, y Resilience4j entrega esa excepción envuelta, sin abrirla, al clasificador. Como la lista de excepciones que cuentan como fallo nombraba la excepción de dominio y no la envoltura, cada caida se contabilizaba como exito.

Solución

Devolver un futuro propio que falla con la excepción original. El efecto fue inmediato: el cortocircuito paso a abrirse al quinto fallo, como estaba configurado.

11.2 Los esquemas que borraban los datos migrados

Síntoma

Al probar la recuperación de un servicio caído, el servicio volvía sano y sin ningún dato.

Causa

Los archivos de esquema terminaban con sentencias de borrado, heredadas de una etapa en que cada servicio se sembraba a si mismo. Como ese archivo se ejecuta en cada arranque, cualquier reinicio -un despliegue, una caida, escalar a más réplicas- se convertía en una perdida de datos silenciosa.

Solución

Retirar las sentencias de borrado. Los datos los pone la migración y se reinician donde corresponde: eliminando el volumen y volviendo a migrar.

11.3 El balanceo que existía pero no se veia

Síntoma

Con tres réplicas levantadas, todas las respuestas declaraban la misma instancia. El reparto parecía inexistente.

Causa

Eran dos problemas encadenados. Las réplicas se registraban en el servicio de descubrimiento con un identificador construido a partir del nombre del servicio y el puerto, identico en las tres, de modo que el registro las tomaba por una sola. Y el campo que identifica a la instancia en cada respuesta se componía del mismo modo, de manera que aunque el reparto hubiera ocurrido, no habría sido visible.

Solución

Incorporar la dirección de red del contenedor tanto al identificador de registro como al campo de la respuesta. Verificado después: tres réplicas atendiendo diez consultas cada una.

12. Propuestas de mejora y próximos pasos

Por orden de importancia, lo que habría que resolver antes de considerar este sistema apto para producción.

- Sacar los secretos del código. Los secretos de los clientes OAuth 2.0 están hoy escritos en una clase de configuración; deben vivir en un gestor de secretos y leerse al arrancar.
- Hacer persistentes las claves de firma. El servidor de autorización genera un par nuevo en cada arranque, de modo que al reiniciarlo los tokens vigentes dejan de validar.
- Cifrado de extremo a extremo con certificados gestionados, al menos entre el exterior y el balanceador.
- Mover los clientes OAuth 2.0 a base de datos. Hoy están en memoria y darlos de alta exige reconstruir la imagen.
- Autenticar el endpoint que dispara la migración, que hoy está protegido solo por no estar publicado.
- Propagar el identificador de correlación que ya genera el gateway hacia los eventos de Kafka, y enviarlo a un recolector de trazas.
- Aumentar el número de particiones por tópico. Con una sola, únicamente una replica de cada grupo consume, de modo que el escalado del lado de los eventos no es efectivo.
- Convertir el proceso batch en una tarea programada en lugar de un servicio permanente, para que no haya nada corriendo entre ejecuciones.

12.1 Camino de despliegue en la nube

El documento de despliegue que acompaña a esta entrega detalla dos caminos. El primero, una instancia de computo ejecutando la misma composición de contenedores, reproduce lo que ya funciona y es el viable dentro de las restricciones del laboratorio académico. El segundo, contenedores gestionados con base de datos y mensajería administradas, es a donde debería ir el sistema en producción.

El salto al segundo camino no exige reescribir código, y eso no es casualidad: es consecuencia de las tres decisiones de arquitectura de la sección 3. Servicios sin estado, configuración por variables de entorno y sondas de salud ya definidas son exactamente lo que un entorno de contenedores gestionados requiere.

13. Conclusión

La solución entregada cubre las tres partes del caso como un sistema único y verificable. Los tres procesos batch del sistema heredado están reescritos con manejo de errores, reintentos y paralelismo, y su resultado alimenta directamente a los microservicios. Los tres canales tienen un backend propio con autenticación y alcances distintos, y la diferencia entre ellos es medible. Los tres microservicios de negocio son dueños exclusivos de su dominio, se comunican por eventos y la única dependencia síncrona que queda está protegida con los cinco patrones de resiliencia.

Más alla de lo que el enunciado pedia, el trabajo dejo tres defectos encontrados y corregidos que comparten una lección: en un sistema distribuido, los fallos más costosos no son los que rompen algo visiblemente, sino los que dejan el sistema aparentando funcionar. Un cortocircuito que nunca se abre, un esquema que borra datos en cada reinicio y un balanceo que no reparte son tres formas distintas del mismo problema, y ninguno se habría encontrado sin probarlos contra el sistema real.

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.